5.4. Les triggers

- **BDA** : Base de Données Active
  - capable de réagir, avec ou sans l'intervention de l'utilisateur, à des événements sur un objet de la base de données.
  - intégration de règles actives : Evénement - Condition – Action
  - implémentation des règles dans des déclencheurs = TRIGGERS
- **TRIGGER** : traitement déclenché par un évenement (opération INSERT, UPDATE, DELETE sur une table)
- **utilité d'un TRIGGER** : intégrité et cohérence des données, mise à jour automatique des données calculées/redondantes, …

```
CREATE TRIGGER   nomtrigger
    BEFORE | AFTER { INSERT | UPDATE | DELETE }
    ON table|vue
    [ FOR EACH ROW ]
    [ WHEN condition ]
    BEGIN
    instructions PLSQL
    END;
```

BEFORE : exécuté avant la mise à jour
AFTER : exécuté après la mise à jour
FOR EACH ROW : exécuté pour chaque ligne mise à jour. Sinon, si clause omise, exécuté une seule fois pour toutes les lignes concernées par la mise à jour (exemple : DELETE FROM table avec la clause : condition du trigger exécuté à la suppression de chaque ligne de la requête)

Les variables OLD et NEW

OLD : référence la ligne avant la mise à jour
NEW : référence la ligne après la mise à jour

opération	OLD	NEW
INSERT	NULL	ligne insérée
UPDATE	ligne avant modification	ligne après modification
DELETE	ligne supprimée	NULL

Exemples

Soit la base de données GESTCOM

article (noart, designation, prixHT, stock)
client (nocli, nomcli, ville)
commande (nocde, datecde, nocli)
detailCde (nocde, noart, qte)

exemple 1 : mise à jour du stock après une insertion dans "detailcde"

```
CREATE TRIGGER MajStockApresInsDetailCde
AFTER INSERT ON detailCde
FOR EACH ROW
    UPDATE article
    SET stock = stock - NEW.qte
    WHERE noart = NEW.noart ;
```

exemple 2 : mise à jour du stock après une suppression dans "detailcde"

```
CREATE TRIGGER MajStockApresSupprDetailCde
AFTER DELETE ON detailCde
FOR EACH ROW
    UPDATE article
    SET stock = stock + OLD.qte
    WHERE noart = OLD.noart ;
```

exemple 3 : mise à jour du stock après une mise à jour de la quantité dans "detailcde"

```
CREATE TRIGGER MajStockApresMajDetailCde
AFTER UPDATE ON detailCde
FOR EACH ROW
    UPDATE article
    SET stock = stock + OLD.qte  - NEW.qte
    WHERE noart = OLD.noart ;
```

exemple 4 : vérification que le stock est suffisant avant insertion dans "detailcde"

```
CREATE TRIGGER VerifStockAvantInsDetailCde
BEFORE INSERT ON detailCde
FOR EACH ROW
BEGIN
    DECLARE stock_insuffisant CONDITION FOR SQLSTATE '45000';
    DECLARE v_stock INT ;
    SELECT stock INTO v_stock FROM article
    WHERE noart = NEW.noart ;
    IF (v_stock < NEW.qte) THEN
        SIGNAL stock_insuffisant  SET MYSQL_ERRNO = 45000,
        MESSAGE_TEXT = 'stock insuffisant ';
    END IF ;
END
```

MYSQL_ERRNO : utile pour la récupération de l'erreur dans un langage hôte.
en PHP , PDO::errorCode() retournera le code '45000' en cas d'erreur